

Laboratório Especial — Trajeto de Aeronaves

Análise do conjunto de dados `flights` (2015)

Pedro Tilhe

16/09/2026

1 Documento compilado em

Este arquivo foi compilado em 16/09/2026 as 09:30:54 (fuso horario America/Sao_Paulo).

2 Objetivo

Este documento implementa a funcao `analisa_aeronave()`, solicitada no enunciado do Laboratorio Especial. Dado o codigo de cauda de uma aeronave (`TAIL_NUMBER`), a funcao:

1. le o arquivo `flights.csv.zip` **por partes** (sem carregar o arquivo inteiro em memoria) e filtra somente os voos daquela aeronave;
2. monta uma tabela *tidy*, ordenada por data e hora de partida, com todas as colunas originais do arquivo `flights.csv.zip` mais algumas colunas derivadas (data/hora de partida, ordem cronologica do voo e velocidade media do voo);
3. usa as coordenadas exatas dos aeroportos (arquivo `airports.csv`) para desenhar, em um mapa, o trajeto completo voado pela aeronave ao longo do ano, respeitando a ordem cronologica dos voos;
4. decora o mapa com duas estatisticas: a **ordem cronologica** do voo (cor do segmento) e a **velocidade media** do voo em milhas por hora, calculada como $\text{DISTANCE} / \text{AIR_TIME} * 60$ (espessura do segmento);
5. retorna uma lista nomeada com a tabela usada no grafico (`tabela`) e o proprio grafico (`grafico`).

3 Sobre o arquivo de aeroportos

O enunciado pede que o mapa utilize “as coordenadas exatas dos aeroportos envolvidos (presentes no arquivo `airports`)”. O arquivo `flights.csv.zip` fornecido nao vem acompanhado

do `airports.csv` oficial do dataset (Kaggle *2015 Flight Delays and Cancellations*), então inclui-se aqui uma cópia desse arquivo (322 aeroportos dos EUA, com colunas `IATA_CODE`, `AIRPORT`, `CITY`, `STATE`, `COUNTRY`, `LATITUDE`, `LONGITUDE`). Ele deve ficar **na mesma pasta** do `flights.csv.zip` — a função o localiza automaticamente a partir do caminho passado em arquivo.

4 Limitação conhecida da base de dados

Ao validar o código com a aeronave `N431WN`, notou-se que os voos de **outubro de 2015** trazem, nas colunas `ORIGIN_AIRPORT` e `DESTINATION_AIRPORT`, um código numérico (identificador de aeroporto do BTS) em vez do código IATA de 3 letras usado nos demais meses do arquivo — uma inconsistência conhecida desta versão do dataset. Como o `airports.csv` só relaciona códigos IATA, esses trechos de outubro não encontram coordenadas e por isso não aparecem desenhados no mapa (a função emite um aviso informando quantos trechos foram afetados). Eles continuam presentes, com todas as suas colunas originais, na tabela retornada — apenas as colunas de latitude/longitude ficam como `NA` nesses casos.

5 A função `analisa_aeronave()`

```
# analisa_aeronave -----
#
# Recebe o código de cauda de uma aeronave e o caminho completo do arquivo
# flights.csv.zip, e retorna o trajeto completo voado por aquela aeronave ao
# longo do ano, em forma de tabela e de mapa.
#
# Argumentos:
#   tail_number : string com o código de cauda da aeronave (ex.: "N431WN")
#   arquivo      : caminho completo para o arquivo flights.csv.zip
#                  (ex.: file.path("dados", "flights.csv.zip")). O arquivo
#                  airports.csv deve estar na mesma pasta que "arquivo".
#
# Retorno:
#   lista nomeada com dois elementos:
#     $tabela : tibble tidy, ordenado por data/hora de partida, com todas as
#               colunas originais de flights.csv.zip mais as colunas
#               derivadas DATA_HORA_PARTIDA, ORDEM_VOO, VELOCIDADE_MEDIA_MPH,
#               LATITUDE/LONGITUDE de origem e destino;
#     $grafico : objeto ggplot com o mapa do trajeto da aeronave.
#
```

```

analisa_aeronave <- function(tail_number, arquivo) {

# --- 1. Localizar e carregar o arquivo de aeroportos -----
# O airports.csv e pequeno (322 linhas), entao pode ser lido de uma vez.
pasta_dos_dados <- dirname(arquivo)
caminho_aeroportos <- file.path(pasta_dos_dados, "airports.csv")

if (!file.exists(caminho_aeroportos)) {
  stop(
    "Nao encontrei o arquivo airports.csv na pasta '", pasta_dos_dados,
    "'. Ele precisa estar na mesma pasta do arquivo de voos."
  )
}

tabela_aeroportos <- readr::read_csv(
  caminho_aeroportos,
  col_types = readr::cols(
    IATA_CODE = readr::col_character(),
    AIRPORT   = readr::col_character(),
    CITY      = readr::col_character(),
    STATE     = readr::col_character(),
    COUNTRY   = readr::col_character(),
    LATITUDE  = readr::col_double(),
    LONGITUDE = readr::col_double()
  )
)

# --- 2. Descobrir o nome do csv dentro do .zip -----
# (o arquivo enviado se chama flights.csv.zip e contem um unico flights.csv,
# mas evitamos supor o nome interno para deixar a funcao mais robusta)
nome_interno_csv <- utils::unzip(arquivo, list = TRUE)$Name[1]

# --- 3. Especificar os tipos de cada coluna -----
# Fazemos isso explicitamente para (a) acelerar a leitura e (b) evitar que
# colunas como TAIL_NUMBER ou os horarios (formato HHMM, ex. "0005") percam
# zeros a esquerda por serem interpretadas como numero.
tipos_das_colunas <- readr::cols(
  YEAR          = readr::col_integer(),
  MONTH         = readr::col_integer(),
  DAY           = readr::col_integer(),
  DAY_OF_WEEK   = readr::col_integer(),
  AIRLINE       = readr::col_character(),

```

```

FLIGHT_NUMBER      = readr::col_integer(),
TAIL_NUMBER        = readr::col_character(),
ORIGIN_AIRPORT     = readr::col_character(),
DESTINATION_AIRPORT = readr::col_character(),
SCHEDULED_DEPARTURE = readr::col_character(),
DEPARTURE_TIME     = readr::col_character(),
DEPARTURE_DELAY    = readr::col_double(),
TAXI_OUT           = readr::col_double(),
WHEELS_OFF         = readr::col_character(),
SCHEDULED_TIME     = readr::col_double(),
ELAPSED_TIME       = readr::col_double(),
AIR_TIME           = readr::col_double(),
DISTANCE           = readr::col_double(),
WHEELS_ON          = readr::col_character(),
TAXI_IN            = readr::col_double(),
SCHEDULED_ARRIVAL  = readr::col_character(),
ARRIVAL_TIME       = readr::col_character(),
ARRIVAL_DELAY      = readr::col_double(),
DIVERTED           = readr::col_integer(),
CANCELLED          = readr::col_integer(),
CANCELLATION_REASON = readr::col_character(),
AIR_SYSTEM_DELAY   = readr::col_double(),
SECURITY_DELAY     = readr::col_double(),
AIRLINE_DELAY      = readr::col_double(),
LATE_AIRCRAFT_DELAY = readr::col_double(),
WEATHER_DELAY      = readr::col_double()
)

# --- 4. Ler o arquivo de voos POR PARTES, mantendo so a aeronave desejada
# Abrimos uma conexao para o csv dentro do zip (sem descompactar em disco)
# e usamos read_csv_chunked: o arquivo e lido em blocos de TAMANHO_DO_BLOCO
# linhas, cada bloco e filtrado pelo tail_number e apenas o resultado fica
# guardado em memoria - o restante do bloco e descartado a cada iteracao.
TAMANHO_DO_BLOCO <- 300000

filtra_pela_aeronave <- function(bloco_de_linhas, posicao) {
  dplyr::filter(bloco_de_linhas, TAIL_NUMBER == tail_number)
}

conexao_csv <- unz(arquivo, nome_interno_csv)

voos_da_aeronave <- readr::read_csv_chunked(

```

```

    file      = conexao_csv,
    callback   = readr::DataFrameCallback$new(filtra_pela_aeronave),
    chunk_size = TAMANHO_DO_BLOCO,
    col_types  = tipos_das_colunas,
    progress   = FALSE
  )

  if (nrow(voos_da_aeronave) == 0) {
    stop("Nenhum voo encontrado para a aeronave '", tail_number, "'.")
  }

  # --- 5. Construir data/hora de partida e ordenar cronologicamente -----
  # SCHEDULED_DEPARTURE vem no formato HHMM (ex.: "0700" = 07:00). Usamos o
  # horario PROGRAMADO de partida (em vez do horario real) porque ele existe
  # mesmo para voos cancelados, o que evita perder trechos do trajeto.
  hora_programada <- stringr::str_pad(voos_da_aeronave$SCHEDULED_DEPARTURE, 4, pad = "0")
  hora_programada <- ifelse(hora_programada == "2400", "0000", hora_programada)
  horas <- as.integer(substr(hora_programada, 1, 2))
  minutos <- as.integer(substr(hora_programada, 3, 4))

  voos_da_aeronave <- voos_da_aeronave |>
    dplyr::mutate(
      DATA_HORA_PARTIDA = lubridate::make_datetime(
        year = YEAR, month = MONTH, day = DAY, hour = horas, min = minutos
      )
    ) |>
    dplyr::arrange(DATA_HORA_PARTIDA) |>
    dplyr::mutate(
      ORDEM_VOO = dplyr::row_number(),
      # velocidade media do voo, em milhas por hora (DISTANCE esta em milhas
      # e AIR_TIME em minutos); voos sem tempo de voo registrado ficam com NA
      VELOCIDADE_MEDIA_MPH = dplyr::if_else(
        !is.na(AIR_TIME) & AIR_TIME > 0,
        DISTANCE / AIR_TIME * 60,
        NA_real_
      )
    )

  # --- 6. Juntar com as coordenadas dos aeroportos de origem e destino ----
  aeroportos_origem <- tabela_aeroportos |>
    dplyr::select(
      ORIGIN_AIRPORT = IATA_CODE,

```

```

    LATITUDE_ORIGEM = LATITUDE,
    LONGITUDE_ORIGEM = LONGITUDE
  )

aeroportos_destino <- tabela_aeroportos |>
  dplyr::select(
    DESTINATION_AIRPORT = IATA_CODE,
    LATITUDE_DESTINO = LATITUDE,
    LONGITUDE_DESTINO = LONGITUDE
  )

tabela_final <- voos_da_aeronave |>
  dplyr::left_join(aeroportos_origem, by = "ORIGIN_AIRPORT") |>
  dplyr::left_join(aeroportos_destino, by = "DESTINATION_AIRPORT")

aeroportos_sem_coordenada <- tabela_final |>
  dplyr::filter(is.na(LATITUDE_ORIGEM) | is.na(LATITUDE_DESTINO)) |>
  dplyr::distinct(ORIGIN_AIRPORT, DESTINATION_AIRPORT)

if (nrow(aeroportos_sem_coordenada) > 0) {
  # No arquivo flights.csv.zip original (Kaggle "2015 Flight Delays and
  # Cancellations"), os voos de OUTUBRO/2015 trazem, por uma inconsistencia
  # conhecida da base, o codigo numerico do aeroporto (Origin/Destination
  # Airport ID do BTS) em vez do codigo IATA de 3 letras usado nos demais
  # meses -- por isso eles nao casam com airports.csv (que so tem codigos
  # IATA). Esses voos permanecem na tabela retornada, mas ficam sem
  # coordenadas e por isso nao aparecem desenhados no mapa.
  trechos_com_codigo_numerico <- sum(
    grepl("[0-9]+$", tabela_final$ORIGIN_AIRPORT) |
    grepl("[0-9]+$", tabela_final$DESTINATION_AIRPORT)
  )
  warning(
    nrow(aeroportos_sem_coordenada), " combinacao(oes) origem-destino ficaram ",
    "sem coordenadas em airports.csv (", trechos_com_codigo_numerico,
    " trecho(s) parecem usar codigo numerico de aeroporto, tipico dos voos ",
    "de outubro/2015 nesta base). Eles continuam na tabela retornada, mas ",
    "nao aparecem no mapa."
  )
}

# --- 7. Montar o mapa do trajeto -----
trechos_para_o_mapa <- tabela_final |>

```

```

dplyr::filter(!is.na(LATITUDE_ORIGEM), !is.na(LATITUDE_DESTINO))

aeroportos_visitados <- dplyr::bind_rows(
  trechos_para_o_mapa |>
    dplyr::transmute(IATA = ORIGIN_AIRPORT, LATITUDE = LATITUDE_ORIGEM, LONGITUDE = LONGITUDE_ORIGEM),
  trechos_para_o_mapa |>
    dplyr::transmute(IATA = DESTINATION_AIRPORT, LATITUDE = LATITUDE_DESTINO, LONGITUDE = LONGITUDE_DESTINO)
) |>
  dplyr::distinct(IATA, .keep_all = TRUE)

mapa_trajeto <- ggplot2::ggplot() +
  ggplot2::borders(
    "state", fill = "grey95", colour = "grey75"
  ) +
  ggplot2::geom_segment(
    data = trechos_para_o_mapa,
    ggplot2::aes(
      x = LONGITUDE_ORIGEM, y = LATITUDE_ORIGEM,
      xend = LONGITUDE_DESTINO, yend = LATITUDE_DESTINO,
      colour = ORDEM_VOO, linewidth = VELOCIDADE_MEDIA_MPH
    ),
    lineend = "round", alpha = 0.85
  ) +
  ggplot2::geom_point(
    data = aeroportos_visitados,
    ggplot2::aes(x = LONGITUDE, y = LATITUDE),
    size = 1.6, colour = "black"
  ) +
  ggplot2::scale_colour_viridis_c(name = "Ordem cronologica\ndo voo no ano") +
  ggplot2::scale_linewidth_continuous(name = "Velocidade media\n(mph)", range = c(0.3, 2.2)) +
  ggplot2::coord_quickmap() +
  ggplot2::labs(
    title = paste("Trajeto da aeronave", tail_number),
    subtitle = paste0(
      nrow(tabela_final), " voos entre ",
      format(min(tabela_final$DATA_HORA_PARTIDA), "%d/%m/%Y"), " e ",
      format(max(tabela_final$DATA_HORA_PARTIDA), "%d/%m/%Y")
    ),
    x = NULL, y = NULL
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(

```

```

    axis.text = ggplot2::element_blank(),
    axis.ticks = ggplot2::element_blank(),
    panel.grid = ggplot2::element_blank()
  )

# --- 8. Retornar a lista nomeada exigida no enunciado -----
list(
  tabela = tabela_final,
  grafico = mapa_trajeto
)
}

```

6 Exemplo de uso

Trocando `tail_number` e `arquivo` abaixo, a função pode ser usada para qualquer aeronave presente no arquivo `flights.csv.zip`. O exemplo abaixo usa a aeronave N431WN, citada no próprio enunciado do laboratório. Este `.qmd` foi salvo na mesma pasta que `flights.csv.zip` e `airports.csv`; se você organizar os dados em uma subpasta (como sugere o enunciado, ex. `file.path("dados", "flights.csv.zip")`), basta ajustar o caminho abaixo — a função sempre procura o `airports.csv` na mesma pasta do arquivo de voos informado.

A leitura do arquivo inteiro (~600 MB descompactados, ~5.8 milhões de linhas) em blocos leva de 1 a 3 minutos, dependendo da máquina.

```

caminho_do_arquivo_de_voos <- "flights.csv.zip"

resultado_n431wn <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo     = caminho_do_arquivo_de_voos
)

```

6.1 Tabela (10 primeiras linhas, colunas mais relevantes)

```

resultado_n431wn$tabela |>
  dplyr::select(
    ORDEM_VOO, DATA_HORA_PARTIDA, AIRLINE, FLIGHT_NUMBER,
    ORIGIN_AIRPORT, DESTINATION_AIRPORT, VELOCIDADE_MEDIA_MPH
  ) |>
  head(10) |>
  knitr::kable()

```


OR- DEM_VOIDA	DATA_HORA_VOIDA	AIR- LINE	FLIGHT_NUM- BER	ORI- GN_AIR- PORT	DESTINA- TION_AIR- PORT	VELOCI- DADE_ME- DIA_MPH
1	2015-01-01 10:35:00	WN	1762	DCA	HOU	374.7692
2	2015-01-01 13:05:00	WN	3564	HOU	BOS	530.4396
3	2015-01-01 17:55:00	WN	3564	BOS	BWI	299.1892
4	2015-01-01 20:20:00	WN	3564	BWI	MCI	329.6591
5	2015-01-02 07:00:00	WN	1793	MCI	LAX	454.3333
6	2015-01-02 09:05:00	WN	1793	LAX	LAS	321.8182
7	2015-01-02 10:40:00	WN	1793	LAS	ICT	488.9256
8	2015-01-02 15:35:00	WN	1793	ICT	DAL	312.1875
9	2015-01-02 17:20:00	WN	1793	DAL	MDW	490.5155
10	2015-01-02 20:05:00	WN	3144	MDW	ATL	412.3256

6.2 Mapa do trajeto

resultado_n431wn\$grafico

Trajeto da aeronave N431WN
1938 voos entre 01/01/2015 e 31/12/2015

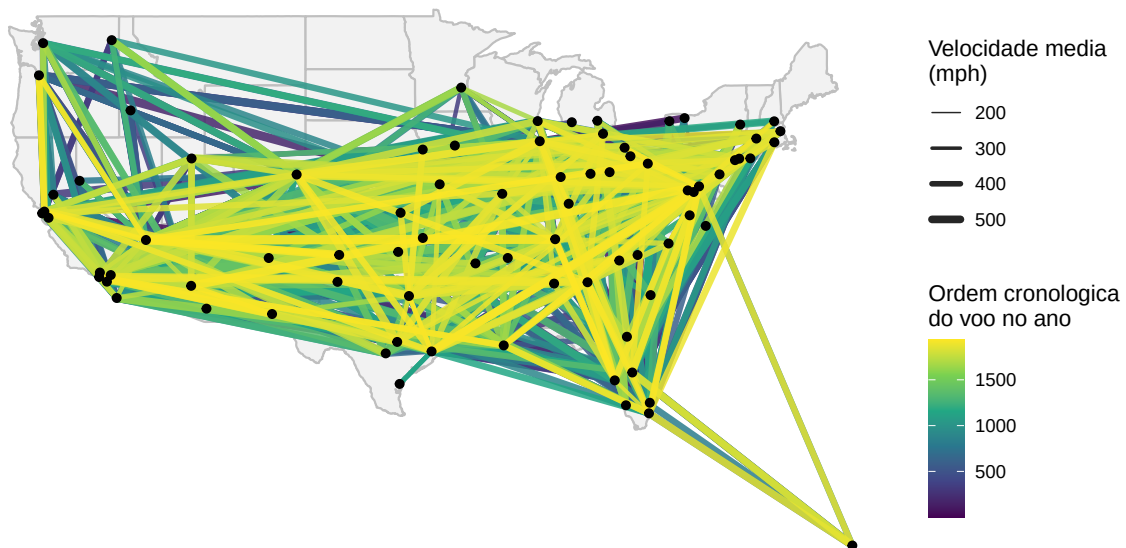


Figure 1: Trajeto cronologico da aeronave N431WN ao longo de 2015.